

This test case exposes the fundamental issue with your approach.

Tree:

```
  4
 /
4
 /
4
```



Let's trace your code.

At the bottom leaf

```
4
```



Returns:

```
4
```



At the middle node

```
  4
 /
4
```



```
</> C++
```



```
l = 4;
r = 0;

check(node, l + r)
check(4, 4) // true
```

So you return

```
l + r + node->data
= 4 + 0 + 4
= 8
```



At the root

```
4
 /
4
 /
4
```



Now

```
</> C++
```



```
l = 8;  
r = 0;
```

```
check(4, 8)
```

This becomes

```
8 == 4 ? ✖
```



So your function returns `false`.

Why is this wrong?

The Sum Property says:

```
node->data = left child value + right child value
```



NOT

```
node->data = left subtree sum + right subtree sum
```



For the root,

```
left child value = 4  
right child value = 0
```



```
4 = 4 + 0 ✓
```

The subtree rooted at the left child has sum `8`, but the problem never asks you to compare subtree sums.

Your recursion computes subtree sums:

```
</> C++  
  
return 1 + r + node->data;
```

So the parent receives

8

instead of the left child's value

4

Conclusion

Your recursion strategy **cannot solve this problem** because the value you return (subtree sum) is different from what the property requires (immediate children's values).

The `-1` propagation was only fixing bugs in your implementation—it doesn't fix the underlying logic.

To solve this problem correctly, you must check at each node:

```
</> C++  
  
left = node->left ? node->left->data : 0;  
right = node->right ? node->right->data : 0;  
  
if (node->data != left + right)  
    return false;
```

and recurse on the left and right subtrees.

So the issue is **not a bug anymore**—it's that the algorithm itself is checking the wrong quantity.